Extending Your Application with Private Microservices

The first part of episode 3 the focuses on extending the microservices architecture by adding a private `fruits` service to the existing setup, which already includes a public `client` service running on Amazon ECS with Fargate. The emphasis is on creating private services accessible only within the VPC, integrating them with the `client` service, and setting the stage for future service mesh adoption. The notes explain the configuration of ECS task definitions, services, load balancers, and security groups, detailing why private services differ from public ones, how to manage internal traffic, and the importance of Terraform state management. The approach builds from the `client` service, reusing patterns to minimize redundancy while introducing new concepts like internal load balancers and environment variable configurations for service communication.

Recap of the Existing Architecture

The architecture so far includes a Virtual Private Cloud (VPC) with public and private subnets, an ECS cluster, and a public `client` service. The `client` service, hosted in private subnets for security, is exposed via an Application Load Balancer (ALB) in public subnets, accessible over HTTP on port 80. The setup uses Fargate for serverless container execution, with task definitions specifying container details and security groups controlling traffic. The goal is to add a private `fruits` service that the `client` can call internally, simulating a backend microservice (e.g., for fruit-related data) while remaining inaccessible externally.

Why Add a Private Service?

Microservices architectures split functionality into specialized services. The `client` service acts as a user-facing entry point, while backend services like `fruits` handle specific tasks (e.g., data retrieval). Private services enhance security by restricting access to internal components, reducing attack surfaces. This chapter replicates the client service pattern for the `fruits` service, adjusting for internal access and connectivity.

Configuring the Fruits Service

The `fruits` service mirrors the `client` service but is private, meaning its ALB resides in private subnets and is only accessible within the VPC. The setup involves an ECS task definition, service, load balancer, and security groups, all defined in Terraform.

ECS Task Definition for Fruits

The task definition outlines the container for the `fruits` service, using the same fake-service image as the `client` for consistency. It specifies Fargate compatibility, modest resource allocations (256 CPU units, 512 MB memory), and `awsvpc` networking for VPC integration. The container listens on port 9090, with environment variables setting its identity and response.

```
# File: ecs-task-definition.tf

resource "aws_ecs_task_definition" "fruits" {
  family                   = "${var.default_tags.project}-fruits"
  requires_compatibilities = ["FARGATE"]
  memory                   = 512
  cpu                      = 256
  network_mode             = "awsvpc"

  container_definitions = jsonencode([
    {
      name      = "fruits"
      image     = "nicholasjackson/fake-service:v0.23.1"
      cpu       = 0
      essential = true
      portMappings = [
        {
          containerPort = 9090
          hostPort      = 9090
          protocol      = "tcp"
        }
      ]
      environment = [
        {
          name  = "NAME"
          value = "fruits"
        },
        {
          name  = "MESSAGE"
          value = "Hello from the fruits client"
        }
      ]
    }
  ])
}
```

Why These Settings?

- **CPU=0**: Allows flexible resource sharing, as in the `client` service, since only one

container runs per task.
- **Port 9090**: Matches the fake-service's default listener, consistent with the `client`.
- **Environment Variables**: Set `NAME` and `MESSAGE` to customize the service's response, simulating a fruit-specific API without code changes.
- **Omitted Upstream URIs**: Unlike the client, the `fruits` service doesn't call other services yet, keeping it simple.

Updating the `client` Task Definition

To enable the `client` to call the `fruits` service, add an `UPSTREAM_URIS` environment variable pointing to the `fruits` ALB's DNS name. This requires updating the client's task definition, which triggers a task replacement during `terraform apply`.

```
# File: ecs-task-definition.tf

resource "aws_ecs_task_definition" "client" {
  family                   = "${var.default_tags.project}-client"
  requires_compatibilities = ["FARGATE"]
  memory                   = 512
  cpu                      = 256
  network_mode             = "awsvpc"

  container_definitions = jsonencode([
    {
      name      = "client"
      image     = "nicholasjackson/fake-service:v0.23.1"
      cpu       = 0
      essential = true
      portMappings = [
        {
          containerPort = 9090
          hostPort      = 9090
          protocol      = "tcp"
        }
      ]
      environment = [
        {
          name  = "NAME"
          value = "client"
        },
        {
          name  = "MESSAGE"
          value = "Hello from the client"
        },
        {
          # added part for upstream
          name  = "UPSTREAM_URIS"
          value = "http://${aws_lb.fruits_alb.dns_name}"
        }
      ]
    }
  ])
```

```
}
```

Why Update the Client?

The `UPSTREAM_URIS` variable tells the fake-service container to forward requests to the `fruits` service's ALB, enabling internal communication. The DNS reference (`aws_lb.fruits_alb.dns_name`) dynamically resolves to the `fruits` ALB, avoiding hardcoding. This update causes Terraform to replace the `client` task to apply the new configuration, demonstrating state management's role in tracking changes.

ECS Service for Fruits

The `fruits` service runs the task definition, maintaining one instance in private subnets for security. It integrates with an internal ALB for traffic routing within the VPC.

```
# File: ecs-services.tf

resource "aws_ecs_service" "fruits" {
  name            = "${var.default_tags.project}-fruits"
  cluster         = aws_ecs_cluster.main.arn
  task_definition = aws_ecs_task_definition.fruits.arn
  desired_count   = 1
  launch_type     = "FARGATE"

  load_balancer {
    target_group_arn = aws_lb_target_group.fruits_alb_targets.arn
    container_name   = "fruits"
    container_port   = 9090
  }

  network_configuration {
    subnets          = aws_subnet.private.*.id
    assign_public_ip = false
    security_groups  = [aws_security_group.ecs_fruits_service.id]
  }
}
```

Why Private Subnets?

Placing the service in private subnets ensures no public access, relying on the ALB for routing. `assign_public_ip = false` prevents external exposure, and the custom security group (defined below) restricts traffic to authorized sources.

Configuring the Internal Load Balancer

The `fruits` service uses an internal ALB in private subnets, unlike the client's public ALB. This restricts access to within the VPC, ideal for backend services.

Fruits ALB Configuration

The ALB is marked `internal = true`, placed in private subnets, and linked to a dedicated security group.

```
# File: ecs-loadbalancers.tf

resource "aws_lb" "fruits_alb" {
  name_prefix        = "fr-"
  load_balancer_type = "application"
  security_groups    = [aws_security_group.fruits_alb.id]
  subnets            = aws_subnet.private.*.id
  idle_timeout       = 60
  internal           = true

  tags = { "Name" = "${var.default_tags.project}-fruits-alb" }
}
```

Why Internal?

The `internal = true` setting ensures the ALB's DNS is only resolvable within the VPC, preventing external access attempts (e.g., via browser, which should hang). This is critical for backend services handling sensitive operations.

Target Group and Listener

The target group registers Fargate tasks by IP, with health checks to ensure only healthy tasks receive traffic. The listener forwards HTTP traffic on port 80 to the target group.

```
# File: ecs-loadbalancers.tf

resource "aws_lb_target_group" "fruits_alb_targets" {
  name_prefix          = "fr-"
  port                 = 9090
  protocol             = "HTTP"
  vpc_id               = aws_vpc.main.id
  deregistration_delay = 30
  target_type          = "ip"

  health_check {
    enabled             = true
    path                = "/"
    healthy_threshold   = 3
    unhealthy_threshold = 3
    timeout             = 30
    interval            = 60
    protocol            = "HTTP"
  }

  tags = { "Name" = "${var.default_tags.project}-fruits-tg" }
}

resource "aws_lb_listener" "fruits_alb_http_80" {
  load_balancer_arn = aws_lb.fruits_alb.arn
  port              = 80
  protocol          = "HTTP"

  default_action {
    type             = "forward"
    target_group_arn = aws_lb_target_group.fruits_alb_targets.arn
  }
}
```

Why These Settings?

- **Target Type IP**: Fargate assigns unique IPs to tasks, unlike EC2's instance-based targeting.
- **Health Checks**: Ensure tasks are responsive at "/", with balanced thresholds to

avoid flapping.
- **Port 80**: Standard for HTTP, forwarding to the container's 9090 port via the target group.

Securing the Fruits Service

Security groups enforce least-privilege access, allowing only necessary traffic to the ALB and ECS service.

Fruits ALB Security Group

Allow inbound HTTP from the `client` service's security group (not public internet) and unrestricted outbound.

```
# File: security-groups.tf

resource "aws_security_group" "fruits_alb" {
  name_prefix = "${var.default_tags.project}-ecs-fruits-alb"
  description = "security group for fruits service application load balancer"
  vpc_id      = aws_vpc.main.id
}

resource "aws_security_group_rule" "fruits_alb_allow_80" {
  security_group_id        = aws_security_group.fruits_alb.id
  type                     = "ingress"
  protocol                 = "tcp"
  from_port                = 80
  to_port                  = 80
  source_security_group_id = aws_security_group.ecs_client_service.id
  description              = "Allow HTTP traffic."
}

resource "aws_security_group_rule" "fruits_alb_allow_outbound" {
  security_group_id = aws_security_group.fruits_alb.id
  type              = "egress"
  protocol          = "-1"
  from_port         = 0
  to_port           = 0
  cidr_blocks       = ["0.0.0.0/0"]
  ipv6_cidr_blocks  = ["::/0"]
  description       = "Allow any outbound traffic."
}
```

Why Restrict to Client Service?

Using `source_security_group_id` ensures only the `client` service's tasks can reach the `fruits` ALB, preventing unauthorized VPC access.

Fruits ECS Service Security Group

Allow inbound from the `fruits` ALB on port 9090, self-referential traffic for potential multi-task setups, and unrestricted outbound.

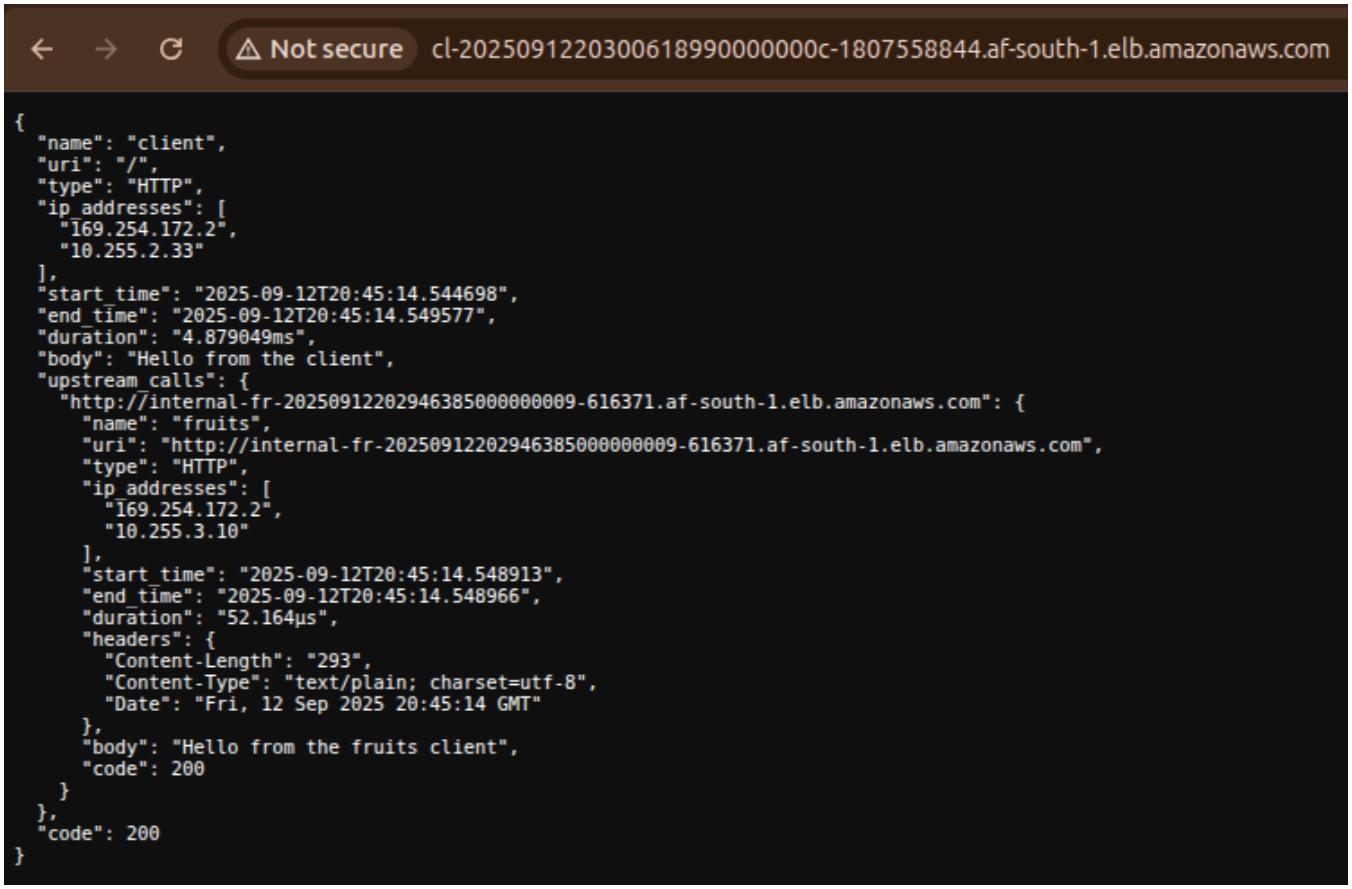
```
# File: security-groups.tf

resource "aws_security_group" "ecs_fruits_service" {
  name_prefix = "${var.default_tags.project}-ecs-fruits-service"
  description = "ECS Fruits service security group."
  vpc_id      = aws_vpc.main.id
}

resource "aws_security_group_rule" "ecs_fruits_service_allow_9090" {
  security_group_id        = aws_security_group.ecs_fruits_service.id
  type                     = "ingress"
  protocol                 = "tcp"
  from_port                = 9090
  to_port                  = 9090
  source_security_group_id = aws_security_group.fruits_alb.id
  description = "Allow incoming traffic from the fruits ALB into the service container por
}

resource "aws_security_group_rule" "ecs_fruits_service_allow_inbound_self" {
  security_group_id = aws_security_group.ecs_fruits_service.id
  type              = "ingress"
  protocol          = -1
  self              = true
  from_port         = 0
  to_port           = 0
  description       = "Allow traffic from resources with this security group."
}

resource "aws_security_group_rule" "ecs_fruits_service_allow_outbound" {
  security_group_id = aws_security_group.ecs_fruits_service.id
  type              = "egress"
  protocol          = "-1"
  from_port         = 0
  to_port           = 0
  cidr_blocks       = ["0.0.0.0/0"]
  ipv6_cidr_blocks  = ["::/0"]
  description       = "Allow any outbound traffic."
}
```

Why Self-Referential Rule?

The `self = true` rule supports future multi-container tasks within the same service communicating internally, though not used here with a single task.

Testing and Validating the Setup

After applying with `terraform apply`, the `client` service's ALB DNS (from `outputs.tf`) should return a response including the `fruits` service's message ("Hello from the `fruits` client") as shown below. The `fruits` ALB's DNS, being internal, should be unreachable externally, confirming security settings.



Note: If issues arise (e.g., tasks draining), check the ECS console for task statuses, ensuring new tasks with updated environment variables are active.

Why Task Replacement Happens

Updating the client task definition (adding `UPSTREAM_URIS`) triggers a service redeployment. Terraform's state file tracks this change, replacing the old task with a new one, which may

cause brief delays as tasks drain and restart.

Adding the Vegetables Service

Building on the private `fruits` service, the next step is to introduce a `vegetables` microservice. This service follows the same architectural and security patterns, ensuring it is only accessible within the VPC and callable by the `client` service. The process involves defining a new ECS task, service, internal load balancer, target group, and security groups in Terraform.

ECS Task Definition for Vegetables

The `vegetables` task definition is nearly identical to `fruits`, using the same container image and resource settings. The main differences are the environment variables, which identify the service as `vegetables`.

```
# File: ecs-task-definition.tf

resource "aws_ecs_task_definition" "vegetables" {
  family                   = "${var.default_tags.project}-vegetables"
  requires_compatibilities = ["FARGATE"]
  memory                   = 512
  cpu                      = 256
  network_mode             = "awsvpc"

  container_definitions = jsonencode([
    {
      name      = "vegetables"
      image     = "nicholasjackson/fake-service:v0.23.1"
      cpu       = 0
      essential = true
      portMappings = [
        {
          containerPort = 9090
          hostPort      = 9090
          protocol      = "tcp"
        }
      ]
      environment = [
        {
          name  = "NAME"
          value = "vegetables"
        },
        {
          name  = "MESSAGE"
          value = "Hello from the vegetables service"
        }
      ]
    }
  ])
}
```

Why Mirror the Fruits Service?

Consistency in configuration simplifies management and troubleshooting, while unique environment variables distinguish the service's identity and response.

Updating the Client Task Definition for Vegetables

To enable the `client` to call both `fruits` and `vegetables`, update the `UPSTREAM_URIS` environment variable to include both internal ALB DNS names, separated by a comma.

```
# File: ecs-task-definition.tf

resource "aws_ecs_task_definition" "client" {
  # ... other settings unchanged ...
  container_definitions = jsonencode([
    {
      # ... other container settings ...
      environment = [
        # ... other environment variables ...
        {
          name  = "UPSTREAM_URIS"
          # May not work with terraform apply this way.
          # Used multi-line here only for readability purposes.
          # Actual code in the repo.
          value = """http://${aws_lb.fruits_alb.dns_name},
                  http://${aws_lb.vegetables_alb.dns_name}"""
        }
      ]
    }
  ])
}
```

Why Multiple Upstreams?

This allows the `client` service to forward requests to both backend services, simulating a more realistic microservices environment.

ECS Service for Vegetables

The ECS service for `vegetables` ensures the task runs in private subnets and is registered with its internal ALB.

```
# File: ecs-services.tf

resource "aws_ecs_service" "vegetables" {
  name            = "${var.default_tags.project}-vegetables"
  cluster         = aws_ecs_cluster.main.arn
  task_definition = aws_ecs_task_definition.vegetables.arn
  desired_count   = 1
  launch_type     = "FARGATE"

  load_balancer {
    target_group_arn = aws_lb_target_group.vegetables_alb_targets.arn
    container_name   = "vegetables"
    container_port   = 9090
  }

  network_configuration {
    subnets          = aws_subnet.private.*.id
    assign_public_ip = false
    security_groups  = [aws_security_group.ecs_vegetables_service.id]
  }
}
```

Why Use a Separate Service?

Each backend microservice is managed independently, allowing for isolated scaling, deployment, and security.

Internal Load Balancer for Vegetables

The `vegetables` service uses its own internal ALB, configured similarly to `fruits` but with unique resource names.

```
# File: ecs-loadbalancers.tf

resource "aws_lb" "vegetables_alb" {
  name_prefix        = "veg-"
  load_balancer_type = "application"
  security_groups    = [aws_security_group.vegetables_alb.id]
  subnets            = aws_subnet.private.*.id
  idle_timeout       = 60
  internal           = true

  tags = { "Name" = "${var.default_tags.project}-vegetables-alb" }
}

resource "aws_lb_target_group" "vegetables_alb_targets" {
  name_prefix          = "veg-"
  port                 = 9090
  protocol             = "HTTP"
  vpc_id               = aws_vpc.main.id
  deregistration_delay = 30
  target_type          = "ip"

  health_check {
    enabled             = true
    path                = "/"
    healthy_threshold   = 3
    unhealthy_threshold = 3
    timeout             = 30
    interval            = 60
    protocol            = "HTTP"
  }

  tags = { "Name" = "${var.default_tags.project}-vegetables-tg" }
}

resource "aws_lb_listener" "vegetables_alb_http_80" {
  load_balancer_arn = aws_lb.vegetables_alb.arn
  port              = 80
  protocol          = "HTTP"
```

```
  default_action {
    type             = "forward"
    target_group_arn = aws_lb_target_group.vegetables_alb_targets.arn
  }
}
```

Why Separate Load Balancers?

Each service's ALB can be managed, secured, and scaled independently, and internal ALBs ensure backend services are not exposed publicly.

Security Groups for Vegetables

Security groups restrict access to the `vegetables` ALB and ECS service, mirroring the `fruits` setup.

```
# File: security-groups.tf

resource "aws_security_group" "vegetables_alb" {
  name_prefix = "${var.default_tags.project}-ecs-vegetables-alb"
  description = "security group for vegetables service application load balancer"
  vpc_id      = aws_vpc.main.id
}

resource "aws_security_group_rule" "vegetables_alb_allow_80" {
  security_group_id        = aws_security_group.vegetables_alb.id
  type                     = "ingress"
  protocol                 = "tcp"
  from_port                = 80
  to_port                  = 80
  source_security_group_id = aws_security_group.ecs_client_service.id
  description              = "Allow HTTP traffic from client service."
}

resource "aws_security_group_rule" "vegetables_alb_allow_outbound" {
  security_group_id = aws_security_group.vegetables_alb.id
  type              = "egress"
  protocol          = "-1"
  from_port         = 0
  to_port           = 0
  cidr_blocks       = ["0.0.0.0/0"]
  ipv6_cidr_blocks  = ["::/0"]
  description       = "Allow any outbound traffic."
}

resource "aws_security_group" "ecs_vegetables_service" {
  name_prefix = "${var.default_tags.project}-ecs-vegetables-service"
  description = "ECS Vegetables service security group."
  vpc_id      = aws_vpc.main.id
}

resource "aws_security_group_rule" "ecs_vegetables_service_allow_9090" {
  security_group_id        = aws_security_group.ecs_vegetables_service.id
  type                     = "ingress"
  protocol                 = "tcp"
```

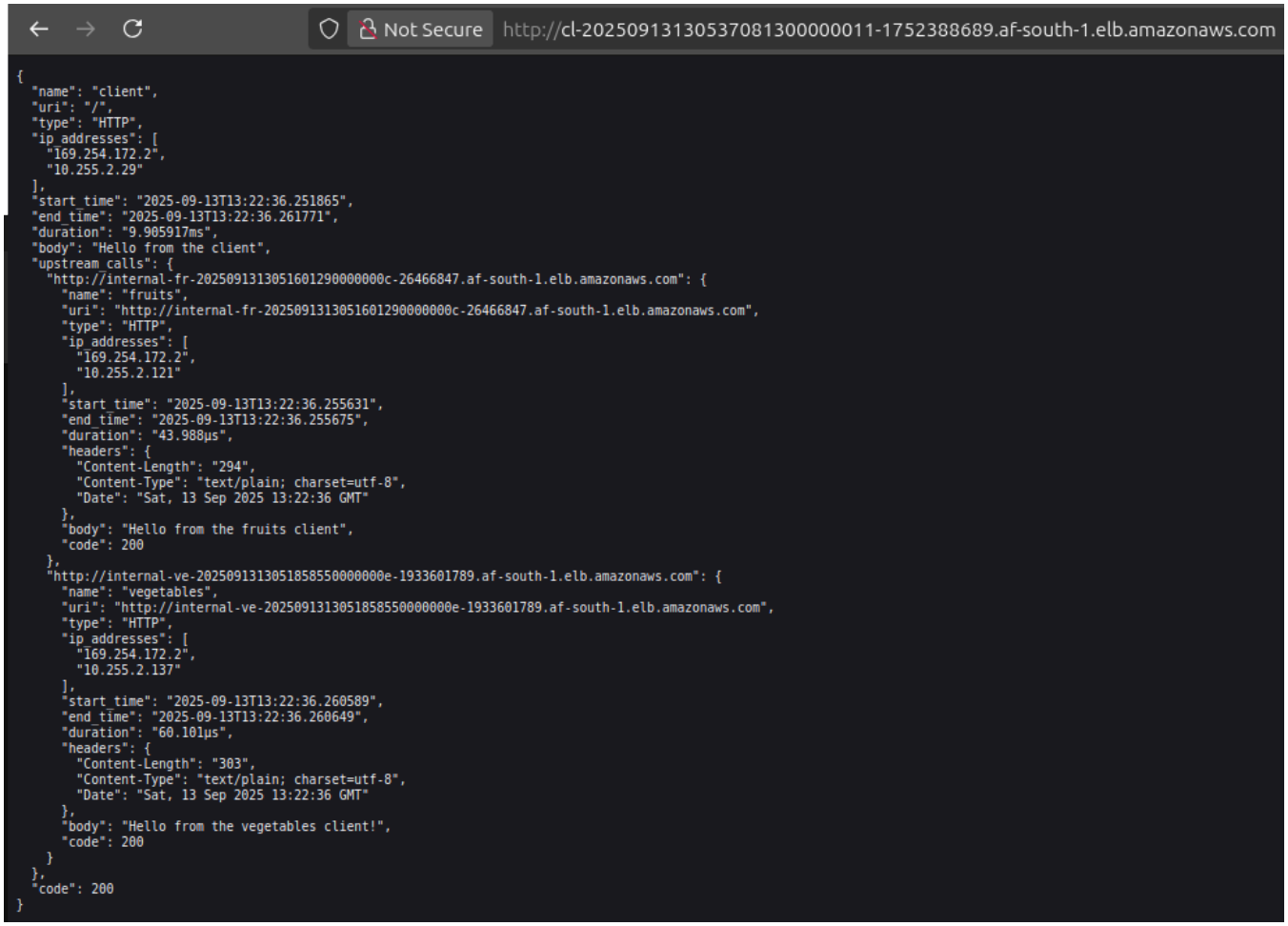
```
  from_port                = 9090
  to_port                  = 9090
  source_security_group_id = aws_security_group.vegetables_alb.id
  description = "Allow incoming traffic from the vegetables ALB into..."
}

resource "aws_security_group_rule" "ecs_vegetables_service_allow_inbound_self" {
  security_group_id = aws_security_group.ecs_vegetables_service.id
  type              = "ingress"
  protocol          = -1
  self              = true
  from_port         = 0
  to_port           = 0
  description       = "Allow traffic from resources with this security group."
}

resource "aws_security_group_rule" "ecs_vegetables_service_allow_outbound" {
  security_group_id = aws_security_group.ecs_vegetables_service.id
  type              = "egress"
  protocol          = "-1"
  from_port         = 0
  to_port           = 0
  cidr_blocks       = ["0.0.0.0/0"]
  ipv6_cidr_blocks  = ["::/0"]
  description       = "Allow any outbound traffic."
}
```

Why Restrict Access?

Only the `client` service can reach the `vegetables` ALB, and only the ALB can reach the ECS service, enforcing strict internal access.

Validating the Vegetables Service

After applying these changes, the `client` service should now be able to call both `fruits` and `vegetables` services. The response from the `client` ALB should include messages from both backends, confirming internal connectivity. The `vegetables` ALB remains inaccessible externally, maintaining security.



Troubleshooting: If the `client` does not display the expected message from `vegetables`, verify task health, security group rules, and that the `UPSTREAM_URIS` environment variable is correctly set.

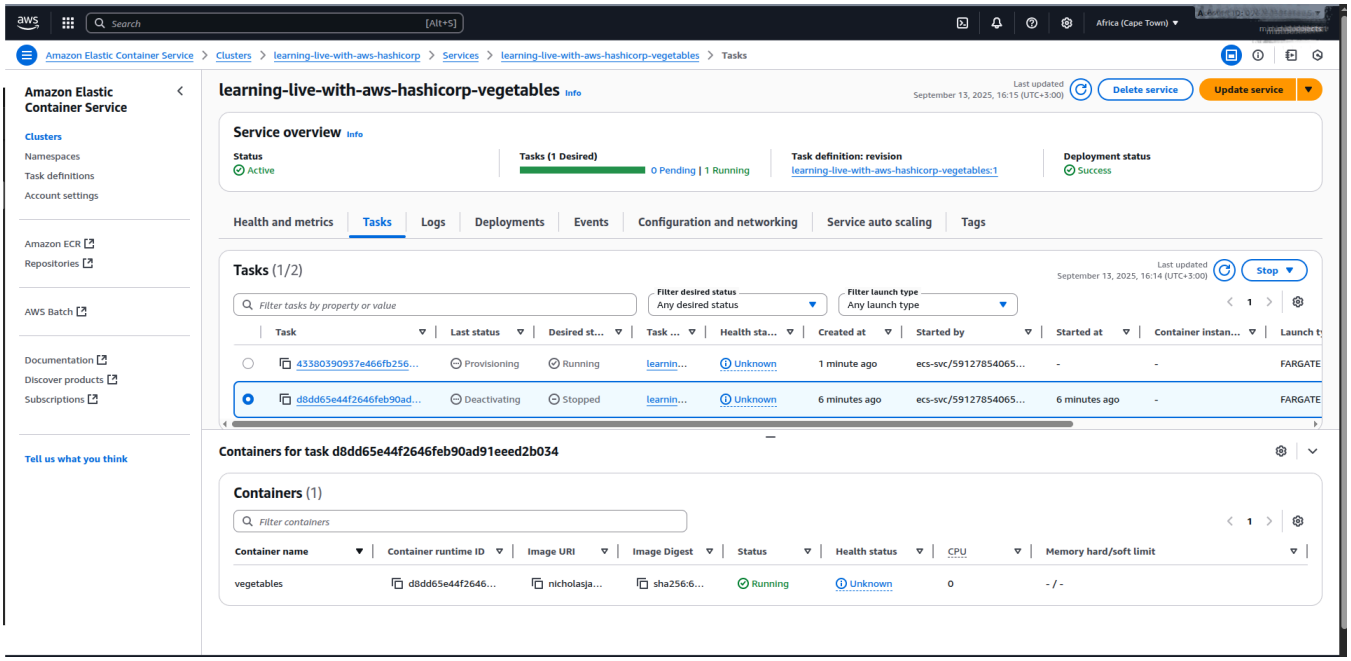
Simulating an Outage on ECS

To understand how ECS and the load balancer handle failures, you can simulate an outage by manually stopping the `vegetables` service task in the ECS console. This action mimics a container crash or application failure.

Steps:

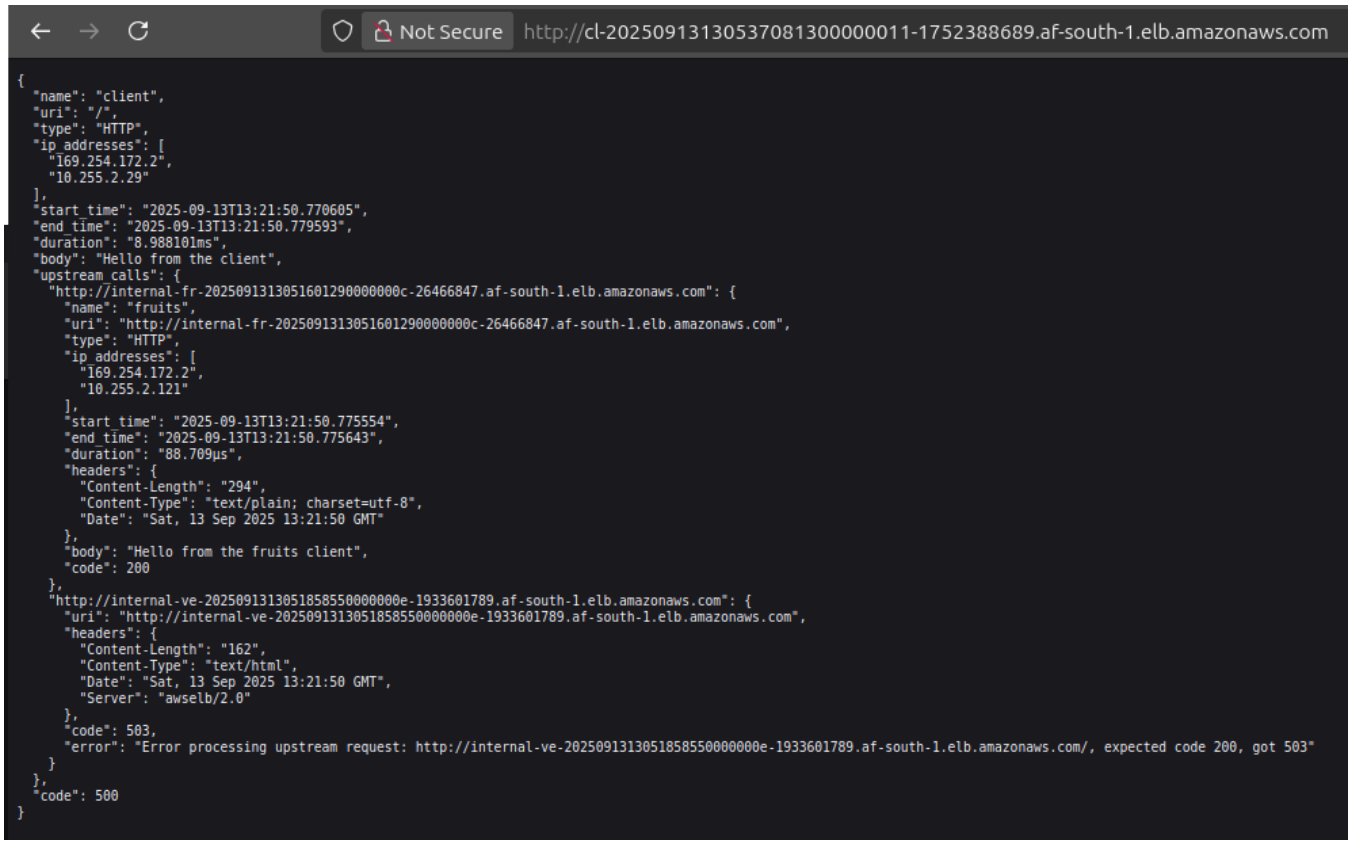
1. Go to the ECS console, select the `vegetables` service, and stop the running task.
2. The service scheduler will automatically launch a new task to maintain the desired count.
3. During the replacement, the ALB health check will detect the missing or unhealthy task and temporarily stop routing traffic to it.

4. Once the new task is healthy, the ALB resumes forwarding requests.



What to Observe:

- The `client` service may briefly fail to reach the `vegetables` backend, but should recover once the new task is running and healthy.
- This demonstrates ECS's self-healing and the importance of health checks for high availability.



Tip: You can also watch the ECS events and ALB target group health status to see the failover and recovery process in real time.

Integrating a Database with Microservices

This section concludes the non-mesh microservices architecture by integrating a simulated database running on an EC2 instance, contrasting with the containerized services on Fargate. It explores why EC2 is chosen for this component, how to dynamically select AMIs using SSM Parameter Store, and configuring user data scripts for automated setup. The narrative builds from the existing fruits and vegetables services, updating them to connect to the database, and introduces security configurations for controlled access. Explanations delve into the differences between EC2 and Fargate, the role of user data in provisioning, and why private IPs and key pairs are essential. Finally, it teases the transition to a service mesh to address scaling challenges like operational overhead and service discovery.

A Database hosted on EC2

To complete the architecture, add a backend database that the fruits and vegetables services

query. Unlike prior services on Fargate, host this on EC2 for variety, simulating a non-containerized component (e.g., a legacy database). This demonstrates hybrid setups where not everything is containerized, setting up for service mesh discussions where unified communication is key.

Why EC2 for the Database?

Fargate abstracts servers, ideal for stateless microservices, but EC2 offers direct control for stateful components like databases requiring persistent storage or custom OS tweaks. Here, use EC2 to run a fake-service as a "database" on a VM, highlighting differences: EC2 needs AMI selection, instance types, and SSH access via key pairs, unlike Fargate's serverless model. This choice illustrates real-world migrations where databases lag containerization.

Prerequisites for EC2 Setup

Before configuring, ensure the VPC, subnets, and prior services (client, fruits, vegetables) are applied. Create an EC2 key pair in the AWS console for SSH (not automated here for security). Define variables for the database's private IP, key pair, name, and message to make the setup reusable.

```
# File: variables.tf (excerpt)

variable "database_private_ip" {
  type        = string
  description = "Private ip address of database"
}

variable "ec2_key_pair" {
  type        = string
  description = "EC2 key pair"
}

variable "database_service_name" {
  type        = string
  description = "Database service name"
  default     = "database"
}

variable "database_message" {
  type        = string
  description = "Database message"
  default     = "Hello from the database"
}
```

Why These Variables?

- **Private IP**: Ensures a static address for reliable internal routing, avoiding DNS resolution overhead.
- **Key Pair**: Enables SSH for debugging or manual interventions, critical for EC2 but absent in Fargate.
- **Defaults**: Provide fallback values for the fake-service, customizable for real databases.

Configuring Terraform Variables

We'll also have to configure Terraform variables in `terraform.tfvars` for the EC2 key pair and the database private IP address. These variables provide reusable, environment-specific values, reducing hardcoding and enabling easy adjustments across deployments.

```
# File: terraform.tfvars

ec2_key_pair        = "learn-live-with-aws-and-hashicorp"
database_private_ip = "10.255.2.253"
```

Why These Variables?

- **ec2_key_pair**: Specifies an SSH key pair for the EC2 instance, enabling secure access for debugging or maintenance. The value `"learn-live-with-aws-and-hashicorp"` must match a key pair created in the AWS console, ensuring SSH connectivity if needed. This is critical for EC2 (unlike Fargate) to manage the VM directly, especially for troubleshooting a non-containerized database.
- **database_private_ip**: Sets a static private IP (`10.255.2.253`) within the VPC's private subnet range for the EC2 instance. A fixed IP ensures consistent addressing for fruits and vegetables services to reach the database without DNS resolution, simplifying configuration and reducing latency. Note that the IP is chosen from the console to avoid conflicts within the subnet (e.g., `10.255.2.0/24`).

Dynamically Selecting an AMI with SSM Parameter Store

AMIs (Amazon Machine Images) define the OS and software for EC2 instances. Hardcoding AMI IDs risks obsolescence; instead, query AWS Systems Manager (SSM) Parameter Store for the latest version dynamically.

Understanding SSM Parameter Store

SSM Parameter Store holds configuration data as parameters, including public ones from AWS for latest AMIs (e.g., Ubuntu versions). It's secure, versioned, and integrates with Terraform via data sources, avoiding manual updates.

Querying the Latest AMI

Use a data source to fetch the Ubuntu 18.04 AMI ID from SSM, ensuring the instance uses a current, patched image.

```
# File: ec2.tf (excerpt)

data "aws_ssm_parameter" "ubuntu1804" {
  name = <<EOT
/aws/service/canonical/ubuntu/server/18.04/stable/current/amd64/hvm/ebs-gp2/ami-id
EOT
}
```

Why SSM Over Hardcoding?
SSM returns the latest AMI ID automatically, promoting security and reducing maintenance. The path targets a specific Ubuntu variant (18.04 LTS, AMD64, HVM, EBS-GP2), but alternatives exist for other distributions or versions.

Configuring the EC2 Instance

The EC2 instance resource provisions the VM in a private subnet, attaching security groups and a key pair. User data scripts automate setup, installing and running the fake-service as a systemd service.

Building the Instance Configuration

Start with basics: AMI from SSM, instance type for cost/performance, subnet for privacy, and tags for organization. Add private IP and key pair for access.

```
# File: ec2.tf

resource "aws_instance" "database" {
  ami                    = data.aws_ssm_parameter.ubuntu1804.value
  instance_type          = "t3.micro"
  subnet_id              = aws_subnet.private[0].id
  vpc_security_group_ids = [aws_security_group.database.id]
  private_ip             = var.database_private_ip
  key_name               = var.ec2_key_pair
  tags                   = { "Name" = "${var.default_tags.project}-database" }

  user_data = base64encode(templatefile("${path.module}/scripts/database.sh", {
    DATABASE_SERVICE_NAME = var.database_service_name
    DATABASE_MESSAGE      = var.database_message
  }))
}
```

Why These Settings?

- **t3.micro**: Balances cost (free-tier eligible) and performance for a simple fake database; upgrade for real workloads.
- **Private Subnet**: Isolates the database, accessible only via internal services.
- **Security Group**: Custom rules (defined below) restrict access to fruits/vegetables services.
- **Private IP and Key Pair**: Static IP for consistent URIs; key pair for SSH troubleshooting.

Automating Setup with User Data

User data runs a Bash script on boot, installing fake-service and configuring it as a systemd service. Encode and template it for variable injection.

```
# File: scripts/database.sh

#!/bin/bash

apt update && apt install -y unzip

# Install Fake Service
curl -LO https://github.com/nicholasjackson/fake-service/releases/download/v0.23.1/fake_se
unzip fake_service_linux_amd64.zip
mv fake-service /usr/local/bin
chmod +x /usr/local/bin/fake-service

# Fake Service Systemd Unit File
cat > /etc/systemd/system/database.service <<- EOF
[Unit]
Description=Database
After=syslog.target network.target
[Service]
Environment="MESSAGE='${DATABASE_MESSAGE}'"
Environment="NAME=${DATABASE_SERVICE_NAME}"
Environment="LISTEN_ADDR=0.0.0.0:27017"
ExecStart=/usr/local/bin/fake-service
ExecStop=/bin/sleep 5
Restart=always
[Install]
WantedBy=multi-user.target
EOF

# Reload unit files and start the database service
systemctl daemon-reload
systemctl start database
```

Why User Data?

It automates provisioning without manual SSH, running as root on first boot. The script installs dependencies, downloads fake-service, and sets up a systemd unit for persistence and restarts. Port 27017 mimics MongoDB, with environment variables customizing the response.

Templating with base64encode: Injects variables (e.g., message) and encodes for AWS transmission, ensuring the script executes correctly.

Updating Services to Connect to the Database

Modify fruits and vegetables task definitions to call the database via its private IP, simulating data queries.

Fruits Task Update

Add `UPSTREAM_URIS` pointing to the database.

```
# File: ecs-task-definition.tf (excerpt)

resource "aws_ecs_task_definition" "fruits" {
  # ... (prior config)

  container_definitions = jsonencode([
    {
      # ... (prior config)

      environment = [
        # ... (prior vars)
        {
          name  = "UPSTREAM_URIS"
          value = "http://${var.database_private_ip}:27017"
        }
      ]
    }
  ])
}
```

Vegetables Task Update

Similarly, add the upstream URI.

```
# File: ecs-task-definition.tf

resource "aws_ecs_task_definition" "vegetables" {
  family                   = "${var.default_tags.project}-vegetables"
  requires_compatibilities = ["FARGATE"]
  memory                   = 512
  cpu                      = 256
  network_mode             = "awsvpc"

  container_definitions = jsonencode([
    {
      # ... (similar to fruits, omitted for brevity)

      environment = [
        # ... (prior vars)
        {
          name  = "UPSTREAM_URIS"
          value = "http://${var.database_private_ip}:27017"
        }
      ]
    }
  ])
}
```

Why Update Tasks?

`UPSTREAM_URIS` configures fake-service to forward requests to the database, enabling chained calls (client → fruits/vegetables → database). The private IP ensures direct, secure internal routing without DNS.

Securing the Database

A dedicated security group restricts access to the database, allowing only fruits and vegetables services.

Database Security Group

Permit inbound on port 27017 from specific service groups and allow all outbound.

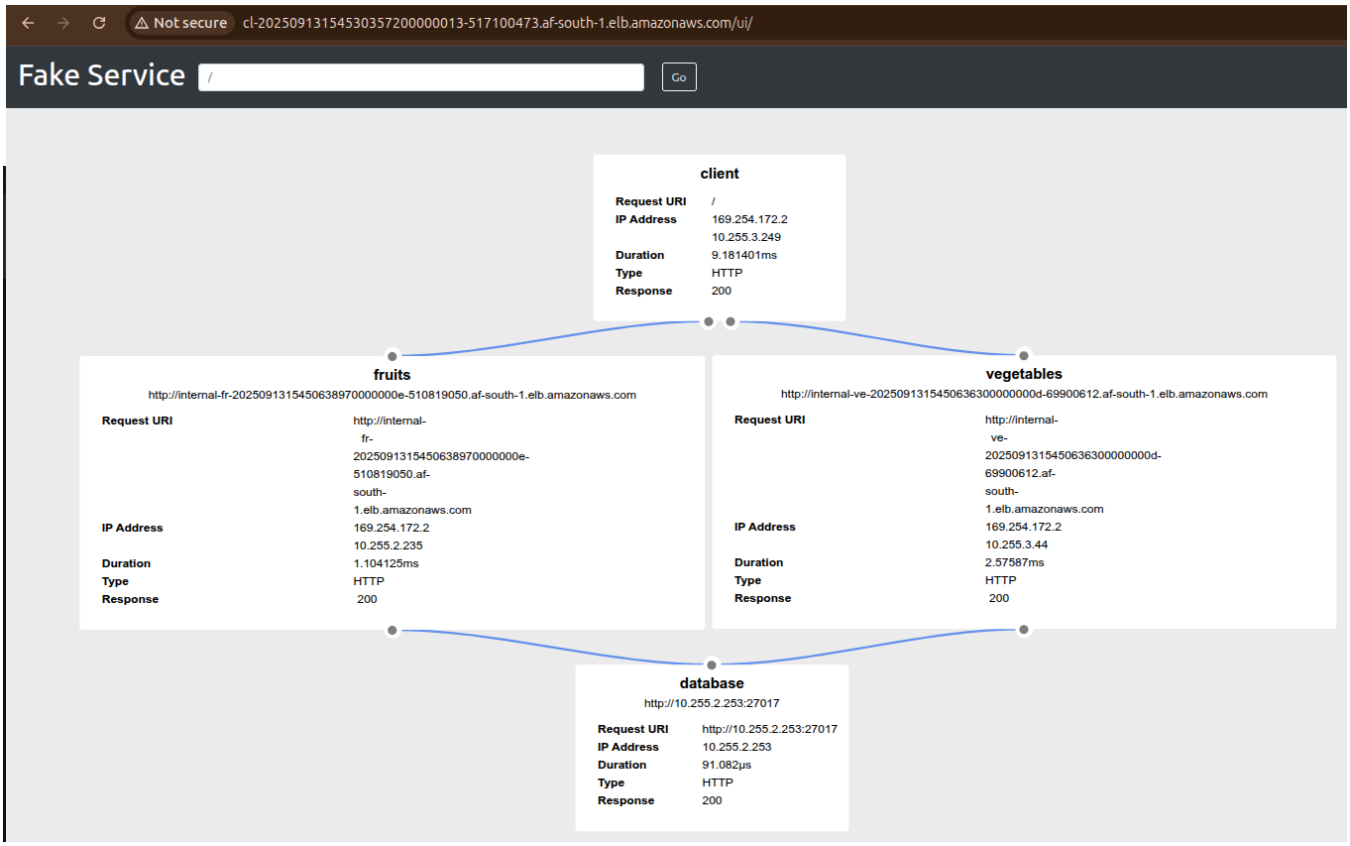
```
# File: security-groups.tf

resource "aws_security_group" "database" {
  name_prefix = "${var.default_tags.project}-database"
  description = "Database security group."
  vpc_id      = aws_vpc.main.id
}

resource "aws_security_group_rule" "database_allow_fruits_27017" {
  security_group_id        = aws_security_group.database.id
  type                     = "ingress"
  protocol                 = "tcp"
  from_port                = 27017
  to_port                  = 27017
  source_security_group_id = aws_security_group.ecs_fruits_service.id
  description              = "Allow incoming traffic from the Fruits service onto the data
}

resource "aws_security_group_rule" "database_allow_vegetables_27017" {
  security_group_id        = aws_security_group.database.id
  type                     = "ingress"
  protocol                 = "tcp"
  from_port                = 27017
  to_port                  = 27017
  source_security_group_id = aws_security_group.ecs_vegetables_service.id
  description              = "Allow incoming traffic from the Vegetables service onto the
}

resource "aws_security_group_rule" "database_allow_outbound" {
  security_group_id = aws_security_group.database.id
  type              = "egress"
  protocol          = "-1"
  from_port         = 0
  to_port           = 0
  cidr_blocks       = ["0.0.0.0/0"]
  ipv6_cidr_blocks  = ["::/0"]
  description       = "Allow any outbound traffic."
}
```

Why Source Security Groups?

Referencing service groups (`source_security_group_id`) ensures only authorized ECS tasks access the database, enforcing least privilege. Outbound allows updates or external calls if needed.

Validating and Testing

After `terraform apply` , the client ALB should chain responses through fruits/vegetables to the database ("Hello from the database") as show below:



If tasks restart, allow drainage time. The database's private nature confirms no external access.

Challenges with the Current Architecture and Path to Service Mesh

As services grow, issues emerge:

- **File Management**: Repetitive code across files increases maintenance.
- **Operational Overhead**: Manual updates (e.g., URIs) for new services introduce errors and ticket workflows.
- **Developer Experience**: Large files hinder focus; independent releases are possible but communication is manual.

A service mesh (e.g., HashiCorp Consul) addresses these by centralizing discovery, reducing load balancers, and automating connectivity, eliminating manual URIs and enhancing observability. Future chapters will implement this for scalable, resilient microservices.